

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Proyecto 2

AURA

Augmented User Responsive Agent

PONDERACIÓN: 40 pts

 **Tiempo estimado: 70 hrs/min**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo.....	4
3. Enunciado del Proyecto.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance del proyecto.....	4
3.3 Entregables.....	5
4. Material de apoyo.....	7
5. Metodología.....	7
6. Recursos y herramientas a utilizar.....	8
7. Desarrollo de Habilidades Blandas.....	8
8. Cronograma.....	9
9. Rúbrica de Calificación.....	10
9.1 Requisitos para optar a la calificación.....	10
9.2 Resumen de Puntuaciones.....	12
9.5 Comentarios Generales.....	15
Detalle de la Calificación.....	15

1. MARCO FORMATIVO

1.1. Valores

Nombre del valor	¿Cómo se aplica en el proyecto?
Integridad Académica	Desarrollo de soluciones propias y originales, respetando la autoría del código y evitando el plagio en la implementación de los modelos lógicos.
Responsabilidad	Cumplimiento estricto de las fechas de entrega y de las especificaciones técnicas del hardware (ejecución exclusiva en Raspberry Pi).
Innovación	Búsqueda de soluciones eficientes para el procesamiento de imágenes y toma de decisiones en hardware de recursos limitados.

1.2. Competencia(s) Técnicas

Competencia	Descripción
Visión Computacional	Capacidad para implementar reconocimiento de patrones, posturas y gestos mediante el uso de OpenCV y MediaPipe.
Integración de APIs y Servicios	Habilidad para comunicar el procesamiento local con plataformas externas de mensajería (Telegram API) y orquestar flujos de trabajo (RPA).
Despliegue en Sistemas Embebidos	Dominio del entorno Linux/Python en arquitecturas ARM (Raspberry Pi) para la ejecución autónoma de modelos de inteligencia artificial.

1.3. Habilidad(es) blandas a formar

Habilidad	Descripción
Resolución de Problemas	Capacidad para depurar código, optimizar el uso de RAM/CPU y resolver conflictos de latencia en el procesamiento de video en tiempo real.
Comunicación Técnica	Habilidad para documentar decisiones arquitectónicas y explicar el flujo de datos entre la percepción, la inferencia y la acción.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Desarrollar un sistema de interacción inteligente humano-robot utilizando Python y una Raspberry Pi para procesar e interpretar gestos humanos en tiempo real, conectando las inferencias a un bot de Telegram y a procesos de RPA, con el fin de desplegar un agente reactivo funcional que deberá entregarse el 16 de octubre de 2026.				

3. Resumen Ejecutivo

El proyecto AURA (Augmented User Responsive Agent) consiste en la creación de un agente inteligente representado mediante un robot virtual 2D, ejecutado sobre una Raspberry Pi y capaz de utilizar visión por computadora para interpretar gestos humanos. El robot virtual funcionará como la representación visual del agente y reacciona de acuerdo con lo que el sistema percibe, interpreta, decide y ejecuta. A través de realidad aumentada, el sistema expondrá su razonamiento interno y tomará decisiones autónomas que podrán desencadenar procesos de Automatización Robótica de Procesos (RPA). Además, permitirá la administración y el control remoto del agente mediante una integración bidireccional con un bot de Telegram.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

En este proyecto deberán desarrollar un sistema de interacción inteligente humano-robot en el que una Raspberry Pi funcione como la unidad central de percepción mediante el uso de una cámara. El sistema debe reconocer personas, poses, gestos o movimientos, y utilizar esta información paramétrica para que un agente interprete el comportamiento humano y tome decisiones autónomas. Implementará una interfaz de Realidad Aumentada que funcionará como una "ventana" hacia el razonamiento del robot: esta interfaz gráfica debe superponerse al video en tiempo real para visualizar qué está percibiendo el sistema (bounding boxes/landmarks), qué interpreta (clasificación del gesto), qué decisión tomó y qué pretende hacer. Como parte de la interfaz de Realidad Aumentada, deberá existir obligatoriamente un **robot virtual 2D que represente visualmente al agente AURA**. Este robot deberá reaccionar a las percepciones, decisiones y acciones del

sistema mediante cambios de estado, expresiones o animaciones. **No se permitirá sustituir el robot virtual únicamente por textos, bounding boxes, landmarks o indicadores gráficos. No se requiere el desarrollo de modelos 3D.**

4.2 Alcance del proyecto

- **Módulo 1: Percepción, Realidad Aumentada y Robot Virtual 2D**

Deberán implementar un motor de reconocimiento continuo que identifique los siguientes eventos y dispare las respuestas asociadas:

Percepción	Interpretación	Respuesta
Persona aparece	Usuario presente	Saludar
Mano levantada	Saludo	Responder
Pulgar arriba	Aprobación	Confirmar
Pulgar abajo	Rechazo	Cambiar respuesta
Señalar izquierda	Dirección	Mostrar opción izquierda
Señalar derecha	Dirección	Mostrar opción derecha
Brazos cruzados	Comando definido	Ejecutar acción
Persona se acerca	Proximidad	Activar interacción
Persona desaparece	Fin de interacción	Despedirse

Robot Virtual 2D

La interfaz deberá incorporar un robot virtual 2D que represente al agente AURA y se encuentre integrado visualmente con el video en tiempo real. El robot deberá reaccionar de acuerdo con los eventos detectados y las decisiones tomadas por el sistema. Como mínimo, deberá representar los siguientes estados:

Esperando → Detectando → Interpretando → Ejecutando → Éxito/Error

El sistema deberá incorporar un **robot virtual 2D que represente visualmente al agente AURA**. Este robot deberá estar compuesto por diferentes assets gráficos (imágenes o sprites 2D) que cambiarán dinámicamente de acuerdo con el estado, percepción, interpretación o acción que esté realizando el agente.

El funcionamiento puede tomar como referencia el comportamiento visual utilizado en **novelas visuales, asistentes virtuales, NPC 2D o mascotas virtuales**, donde un mismo personaje dispone de diferentes poses y expresiones que son mostradas según su estado.

Por ejemplo, juegos como *Doki Doki Literature Club* utilizan diferentes sprites de un personaje para representar cambios de expresión o comportamiento.

Como mínimo, AURA deberá contar con representaciones visuales diferenciadas para los estados de **espera, saludo, detección, interpretación, ejecución, éxito, error y despedida**.

No se requiere animación cuadro por cuadro ni modelado 3D. Se permite utilizar imágenes, sprites u otros assets 2D. **No será válido utilizar un único asset estático y representar los cambios de estado únicamente mediante texto.**

No se permite el uso de robots o modelos 3D.

Ejemplos:

- ❖ Novelas visuales: un personaje cambia de sprite según esté neutral, feliz, sorprendido, preocupado, etc.
- ❖ Asistentes virtuales 2D: un avatar cambia de pose o expresión dependiendo de si está escuchando, procesando, ejecutando o respondiendo.
- ❖ NPC 2D de videojuegos: el personaje utiliza diferentes sprites para representar estados como idle, alerta, interacción, éxito o error.
- ❖ Tamagotchi/mascotas virtuales: el mismo personaje cambia visualmente dependiendo de su estado.
- ❖ Doki Doki Literature Club: como referencia específica de personajes que cambian poses y expresiones mediante distintos assets.

- **Módulo 2: Interfaz de Administración por Telegram**

Deberán programar y conectar un Bot de Telegram que consuma el estado del agente y permita administrar su configuración. Los comandos obligatorios son:

- **/status:** Mostrar CPU, RAM y estado de los servicios de la Raspberry Pi.
- **/iniciar:** Iniciar el sistema de reconocimiento e interacción.
- **/apagar:** Detener el reconocimiento de gestos o apagar controladamente el servicio.
- **/history:** Consultar historial de gestos detectados (fecha, confianza y acción ejecutada).
- **/gestures:** Listar gestos registrados en el sistema.
- **/addGesture:** Registrar/entrenar una nueva señal.
- **/deleteGesture:** Eliminar/deshabilitar una señal.
- **/actions:** Mostrar acciones disponibles.
- **/addAction:** Crear una acción que pueda ejecutar el agente.
- **/linkAction:** Asociar un gesto con una acción determinada.
- **/sendMessage:** Configurar un mensaje de Telegram como respuesta a determinado gesto.
- **/rpa:** Consultar procesos RPA disponibles.
- **/logs:** Mostrar últimos eventos, errores y acciones ejecutadas.

- **Módulo 3: Automatización Robótica de Procesos (RPA)**

El sistema deberá integrar procesos de **Automatización Robótica de Procesos (RPA)** que serán ejecutados a partir de los gestos previamente reconocidos e interpretados por el agente inteligente.

Cada gesto deberá estar asociado a una acción específica. Una vez reconocido, el sistema deberá desencadenar automáticamente el proceso RPA correspondiente y comunicar al usuario el resultado de la operación.

Como mínimo, se deberán implementar los siguientes procesos:

Proceso RPA	Gesto	Significado
Consultar horario	👉 Índice hacia arriba	Consultar / mostrar información
Descargar material	👇 Índice hacia abajo	Descargar
Completar formulario	✌ Simular escritura con el índice	Ingresar / completar información

1. **Consultar horario:**

Al reconocer el gesto asociado, el RPA deberá ingresar al portal académico correspondiente, navegar hasta la información del curso y obtener el horario asignado tanto para la clase magistral como para el laboratorio. La información obtenida deberá ser presentada al usuario por medio del asistente.

2. **Descargar material:**

Al reconocer el gesto asociado, el RPA deberá ingresar a UEDI, localizar el recurso indicado y descargar automáticamente el enunciado solicitado. Al finalizar el proceso, el sistema deberá informar al usuario si la descarga se realizó correctamente.

3. **Completar formulario:**

Al reconocer el gesto asociado, el RPA deberá acceder a un formulario proporcionado por los auxiliares y completar automáticamente los campos solicitados utilizando la información disponible en el sistema. El proceso deberá validar que los campos requeridos hayan sido completados correctamente antes de finalizar.



Alcance obligatorio

El proyecto deberá implementar en su totalidad los módulos 1, 2 y 3 descritos en el enunciado. Los tres módulos deberán encontrarse integrados, permitiendo que la información generada por un módulo pueda desencadenar acciones en los demás cuando corresponda.

Alcance opcional

Los estudiantes podrán implementar funcionalidades adicionales mediante la incorporación de nuevos gestos y su asociación con otros procesos RPA. Estas funcionalidades deberán mantener el flujo definido en el proyecto: **reconocimiento del gesto** → **interpretación de la intención** → **ejecución de la acción** → **presentación del resultado**.

4.3 Recursos y herramientas a utilizar

Recurso	Descripción	Restricción
Hardware	Raspberry Pi (cualquier modelo) y Cámara (cualquier modelo o tipo).	Todo el código debe correr nativamente en el dispositivo; prohibido el procesamiento en la nube para la visión.
Lenguaje	Python 3.x	Uso exclusivo de Python para el desarrollo del backend de IA y control.
Librerías de Visión	OpenCV, MediaPipe	Son las librerías recomendadas para el procesamiento de imágenes

		y extracción de landmarks.
Librería de Mensajería	Telegram Bot API / python-telegram-bot / Por HTTP nativo	Uso exclusivo para la comunicación y administración del agente.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción	Formato
Código Fuente	Repositorio completo con el código del agente, dependencias (requirements.txt) y scripts de ejecución.	Enlace a Repositorio Git (GitHub)
Documentación Técnica	Manual de arquitectura, diagrama de flujo de datos, explicación de los procesos RPA y manual de usuario del Bot.	Archivos Markdown

GIT SECCIÓN A:

Tutor 1: roberto1206

Tutor 2: JavierB20

Nombre del Repositorio

[IA1]Proyecto2_GRUPO#_2S2026_SECA

Estudiantes del tutor 1:

Entregar todos el proyecto de forma individual en uedi

Estudiantes del tutor 2:

Entregar todos el proyecto de forma individual tanto en classroom como en uedi

5. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

- **Paso 1 Configuración del Entorno Embebido:**
 - Instalen el sistema operativo en Raspberry Pi, configuran el entorno virtual de Python e instalen OpenCV y MediaPipe. Verifiquen el acceso a la cámara y monitoreen el uso de CPU/RAM.
- **Paso 2 Implementación de Visión y Extracción de Landmark:**
 - Desarrollen el script base para capturar el video en tiempo real. Utilicen MediaPipe para extraer los puntos clave (landmarks) del cuerpo y las manos.
- **Paso 3 Desarrollo del Módulo de Clasificación de Gestos:**
 - Crean la lógica matemática o heurística para traducir las coordenadas de los landmarks en los gestos requeridos (ej. cálculo de ángulos para detectar brazos cruzados o posiciones relativas para el pulgar arriba/abajo).
- **Paso 4: Construcción de la Interfaz de Realidad Aumentada y Robot Virtual 2D:**
 - Utilicen las funciones gráficas de OpenCV y los recursos 2D necesarios para sobreponer en el video en tiempo real los bounding boxes, landmarks, gesto detectado, nivel de confianza, interpretación y acción a ejecutar. Además, deberán integrar el **robot virtual 2D**, cuyo estado visual deberá cambiar dinámicamente de acuerdo con el proceso que esté realizando el agente: esperando, detectando, interpretando, ejecutando, éxito o error.
- **Paso 5 Integración del Bot de Telegram:**
 - Creen el bot en BotFather, configuren el token e implementen los handlers para los comandos requeridos, asegurando la comunicación de doble vía con el script principal.
- **Paso 6 Orquestación RPA:**
 - Programen los scripts de automatización
- **Paso 7 Pruebas de Estrés y Depuración:**
 - Ejecuten el sistema completo, monitoreen el comando /status para evaluar fugas de memoria, ajusten los umbrales de detección y graben las demostraciones.

6. Material de apoyo y Documentación

Recurso	Tipo	Enlace / Ubicación
Documentación OpenCV Python	Manual Técnico	https://docs.opencv.org/
Guía de MediaPipe Solutions	Manual Técnico	https://developers.google.com/mediapipe

Telegram Bot API	Documentación Oficial	https://core.telegram.org/bots/api
------------------	-----------------------	---

7. Cronograma

Fase	Actividad	Fecha de Inicio	Fecha de Fin
Asignación de Proyecto	Lectura del proyecto	11/09/2026	11/09/2026
Fase 1	Planificación, armado de hardware y configuración del entorno Linux en Raspberry Pi.	11/09/2026	15/09/2026
Fase 2	Desarrollo de Visión Computacional, integración de MediaPipe y detección de gestos (AR).	16/09/2026	29/09/2026
Fase 3	Desarrollo de Bot de Telegram, handlers y vinculación lógica de comandos.	30/09/2026	08/10/2026
Fase 4	Implementación de flujos RPA y estabilización del consumo de recursos del sistema.	09/10/2026	14/10/2026
Fase 5	Pruebas integrales, grabación del video demostrativo y redacción de documentación.	15/10/2026	16/10/2026
Calificación	Evaluación técnica, revisión de código y defensa oral con el equipo.	17/10/2026	17/10/2026

8. Rúbrica y Reglamento de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Motivo de Penalización	Castigo / Puntos a Restar
Ejecución fuera de la Raspberry Pi	Rechazo inmediato (- 100%). Todo procesamiento de IA debe ser embebido localmente.
Uso de librerías de IA no autorizadas	Rechazo inmediato (- 100%). Solo se permite OpenCV y MediaPipe.
Plagio de código o entrega de proyectos de semestres anteriores	Rechazo inmediato (0 puntos en el proyecto) y reporte a coordinación.
Comando de Telegram faltante o defectuoso	-5% por cada comando de la lista obligatoria que falle o no esté implementado.
Falta de Overlays (Realidad Aumentada visual)	-30% puntos si el video no muestra el texto/cajas de razonamiento del agente.

9.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades	80	
Realidad Aumentada	50	
Telegram	20	

RPA	10	
2. Conocimiento	20	
Documentación	10	
Preguntas	10	
TOTAL	100	

*La calificación debe incluir ambas áreas conocimientos y habilidades.

9.5 Comentarios Generales

Detalle de la Calificación

Criterio de Evaluación	Punteo Máx	Satisfactorio (100% - 61%)	Necesita Mejorar (60% - 0%)	Rango Satisfactorio	Rango Necesita Mejorar
1. Habilidades	80				
.1 Ejecución de Realidad Aumentada (Visualización)	50	Renderiza de forma fluida el feed de video superponiendo bounding boxes, landmarks, e indicadores de estado, interpretación y acción	No logra sobreponer los indicadores visuales solicitados o el feed de video es inexistente.	50-25	24 - 0

Criterio de Evaluación	Punteo Máx	Satisfactorio (100% - 61%)	Necesita Mejorar (60% - 0%)	Rango Satisfactorio	Rango Necesita Mejorar
		de forma clara y sin lag evidente.			
1.2 Integración y Control por Telegram (Operatividad)	20	El bot funciona bidireccionalmente sin errores. Ejecuta acciones de sistema como <code>/status</code> o <code>/apagar</code> exitosamente interactuando directamente con el OS de la Raspberry Pi.	El bot no está conectado o sus respuestas están simuladas de manera estática y no responden al estado real del agente.	20 - 10	9 - 0
1.3 Implementación de RPA (Automatización)	10	Ejecuta satisfactoriamente las acciones automatizadas en respuesta a los gestos: el sistema consulta el clima en tiempo real, elabora saludos dinámicos y envía notificaciones por Telegram.	Los procesos RPA no se desencadenan o la aplicación falla al intentar enviar datos complejos a Telegram.	10-5	4-0
2. Conocimientos	20				
2.1 Documentación Técnica	10	Entrega un manual completo que detalla la arquitectura, diagramas de flujo y explicación algorítmica clara sobre la detección espacial de landmarks.	Documentación pobre, desordenada o que no refleja el código realmente entregado.	10-5	4-0
2.2 Defensa de Proyecto (Preguntas)	10	Responde con total seguridad y dominio técnico a las preguntas sobre el funcionamiento de MediaPipe, el uso de recursos en Linux y la orquestación de código.	Desconoce el funcionamiento interno del código presentado o es incapaz de explicar las librerías utilizadas.	10-5	4-0

